

# Practical Work 1: TCP File Transfer

Nguyễn Tmg Bách  
Student ID: 23BI14057

November 21, 2025

## 1 Goal

The goal of this practical work is to implement a one-to-one file transfer application over TCP/IP in the command-line interface. The system consists of one server and one client using the socket API in C.

## 2 Protocol Design

The protocol between client and server is very simple:

1. The client establishes a TCP connection to the server.
2. The client sends a 4-byte unsigned integer in network byte order representing the file size in bytes.
3. The client sends exactly `file_size` bytes of file data.
4. The server receives the file size and then reads file data until all bytes have been received. The server writes the data to an output file.

This design follows the classical client/server flow shown in the lecture slides: `socket()`, `bind()`, `listen()`, `accept()`, `connect()`, `read()`, `write()`, and `close()`.

## 3 System Organisation

The system contains two programs:

- **Server** (`server.c`): listens on a TCP port, accepts one incoming connection and receives a file.
- **Client** (`client.c`): connects to the server and sends a file read from disk.

The compilation is managed by a simple `Makefile`. Figure 1 summarises the control flow.

| Client                 | Server                                  |
|------------------------|-----------------------------------------|
| <code>socket()</code>  | <code>socket()</code>                   |
| <code>connect()</code> | <code>bind(), listen(), accept()</code> |
| send size, send data   | recv size, recv data                    |
| <code>close()</code>   | <code>close(), close()</code>           |

Figure 1: Client/server session for file transfer.

## 4 Implementation Details

The main implementation points are:

- Use of `setsockopt()` with `SO_REUSEADDR` on the server to avoid “address already in use” errors.
- Use of `htonl()` and `ntohl()` to ensure the file size is transmitted in network byte order.
- A helper function `send_all()` on the client to guarantee all bytes are written to the socket.
- A loop on the server side to receive chunks of data until the announced file size is fully received.

Code snippets for the critical parts of the implementation are shown in Listings 1 and 2.

### Client Snippet

Listing 1: Sending the file size and data

```
uint32_t file_size = (uint32_t) fsize;
uint32_t size_net = htonl(file_size);
send_all(sockfd, &size_net, sizeof(size_net));

while ((nread = fread(buffer, 1, BUF_SIZE, fp)) > 0) {
    send_all(sockfd, buffer, nread);
}
```

### Server Snippet

Listing 2: Receiving the file

```
uint32_t size_net;
recv(connfd, &size_net, sizeof(size_net), MSG_WAITALL);
uint32_t file_size = ntohl(size_net);

while (received < file_size) {
    size_t to_read = (file_size - received > BUF_SIZE)
        ? BUF_SIZE : file_size - received;
    n = recv(connfd, buffer, to_read, 0);
    fwrite(buffer, 1, n, fp);
    received += n;
}
```

## 5 Work Division

This practical was done individually. The tasks were:

- Protocol design and system organisation.
- Implementation of client and server in C.
- Testing the application with different file sizes.
- Writing this report in  $\text{\LaTeX}$ .